

PROJECT: Mobile applications-Interaction design in vehicle

Smart Car Dashboard

Project Description Report

Muhammad Ahmed Ali

12500486

Technische Hochschule Deggendorf

Project Workshop — Mobile Applications: Interaction Design in
Vehicle

July 2026

1. Introduction

Smart Car Dashboard is an Android application that connects to live vehicle data over MQTT and displays speed, RPM, and fuel level at a glance, using a car-friendly, high-contrast interface. When the driver exceeds a safe speed, the app responds immediately with a spoken warning and a visual alert banner, so the driver does not need to look away from the road for long. A live weather card keeps the driver informed of current conditions.

This document describes the application, its intended usage, target users, and key use cases, in line with the project requirements for the Automotive Interaction project workshop.

2. Application Description

The application is built around a real-time data pipeline: a simulated On-Board Diagnostics (OBD) source publishes vehicle sensor values (speed, RPM, fuel level) to an MQTT broker, and the Android app subscribes to these values and renders them on a dashboard-style interface.

2.1 Core Features

- Live speed gauge — animated speedometer with color-coded zones (green/yellow/red)
- RPM and fuel display — compact live-updating cards
- Voice speed warning — spoken alert (text-to-speech) when the driver exceeds the speed limit
- Visual alert banner — a red banner appears briefly alongside the voice warning
- Weather card — live temperature and conditions for the driver's location
- Automatic MQTT reconnection — the app recovers automatically if the connection drops

2.2 Communication Protocol

The application uses MQTT (Message Queuing Telemetry Transport), a lightweight publish/subscribe protocol well suited to sensor data. The Android app subscribes to three topics:

- car/speed — current vehicle speed in km/h
- car/rpm — current engine RPM
- car/fuel — current fuel level as a percentage

Each message is published as a small JSON payload, e.g. `{"value": 87.5}`, which keeps the protocol simple, human-readable, and easy to extend with additional fields in the future.

3. Usage

On launch, the app automatically attempts to connect to the configured MQTT broker. Once connected, it subscribes to the sensor topics and begins updating the dashboard in real time. No manual configuration is

required from the end user during normal operation — the app is designed to be glanceable and require zero interaction while driving.

If the vehicle's speed exceeds a configured threshold, the app automatically triggers a spoken warning and displays a temporary red alert banner at the top of the screen. Both cues fire together and are subject to a cooldown period to avoid repeated alerts while the driver remains above the limit.

4. Target User

The application is designed primarily for the following users:

- Everyday drivers who want at-a-glance vehicle information without diverting attention from the road
- Drivers on unfamiliar roads or in areas with variable speed limits, who benefit from an automatic speed-awareness check
- Owners of older or budget vehicles that lack a built-in smart dashboard or voice-assist features

The interface is intentionally high-contrast and large-text, catering to quick glances rather than sustained reading, which is appropriate for use while the vehicle is in motion.

5. Use Cases

5.1 Use Case: Daily Commute with Variable Speed Limits

A driver, Anna, commutes daily on unfamiliar roads with variable speed limits. She starts her car, and the app automatically connects over MQTT and begins showing live speed, RPM, and fuel. As she drives, the dashboard updates continuously, so she does not need to check her phone. When she drifts over the speed limit, a spoken warning and a red banner alert her immediately. A glance at the weather card shows rain is approaching, so she adjusts her driving accordingly.

5.2 Use Case: Speed Awareness Alert

While driving, the vehicle's simulated speed sensor reports a value above the configured speed limit. The Android app receives this value over MQTT within roughly one second, updates the speedometer to a red color state, triggers a spoken warning ("You are driving too fast"), and displays a red alert banner for approximately three seconds before it fades out automatically.

5.3 Use Case: Connection Recovery

If the MQTT connection is temporarily lost (e.g. due to a WiFi drop), the app displays a "Connection lost, retrying..." status message and automatically attempts to reconnect without requiring the user to restart the application.

6. System Architecture

The system consists of four main components that communicate over MQTT:

- OBD Simulator (Python) — publishes simulated speed, RPM, and fuel values every second
- MQTT Broker (Mosquitto) — routes sensor messages between the simulator and the app
- Android App (Kotlin) — subscribes to sensor topics and renders the dashboard UI
- Driver — views the display and hears voice alerts when applicable

Supporting services include a Text-to-Speech engine for spoken warnings, the OpenWeatherMap API for live weather data, and Git for source control of both the Android app and the Python simulator.

7. Implementation

7.1 Technology Stack

- Kotlin, Android Studio (application development)
- Eclipse Paho MQTT client (Android MQTT connectivity)
- Mosquitto (local MQTT broker for testing)
- Python 3 with paho-mqtt (OBD data simulator)
- Android TextToSpeech API (voice alerts)
- OpenWeatherMap REST API (live weather data)

7.2 Key Components

- MainActivity — manages the MQTT client, message handling, and alert logic
- SpeedometerView — a custom Canvas-drawn gauge with smooth animation and color zones
- obd_simulator.py — publishes speed/RPM/fuel data once per second
- Debounce logic — prevents repeated alerts by enforcing an 8-second cooldown

7.3 Notable Implementation Challenges

- Resolving an AppCompatActivity theme mismatch that caused an activity crash on launch
- Configuring Mosquitto's "local only mode" restriction for real-device testing
- Handling emulator networking (10.0.2.2) versus real-device network IP addressing
- Running the weather API call asynchronously to avoid blocking the main UI thread

8. Installation and Setup

To run the project locally, the following steps are required:

- Install Android Studio and open the project in the android-app directory
- Install Mosquitto and start the broker (mosquitto -v)
- Install Python 3 and the paho-mqtt package (pip install paho-mqtt)

-
- Run the OBD simulator script (python obd_simulator.py)
 - Set the broker URL in MainActivity.kt (10.0.2.2 for the emulator, or the host machine's LAN IP for a real device)
 - Build and run the Android app from Android Studio

9. Evaluation Plan

The application will be evaluated along two dimensions, consistent with the project's grading criteria: functional correctness and usability.

9.1 Functional Testing

- Live demonstration with the simulator running and dashboard values updating in real time
- Manually publishing a high-speed value to confirm the voice warning and banner trigger correctly
- Verifying the weather card loads and refreshes as expected
- Testing reconnection behavior after a simulated network interruption

9.2 Usability Evaluation

A short Google Forms survey will be used to gather feedback from 3–5 test users, covering dashboard clarity, readability, alert noticeability, and overall satisfaction. Results will be summarized and used to identify any final adjustments before submission.

10. Git Repository

All source code, including the Android application and the Python OBD simulator, is version controlled in a single Git repository with the following structure:

dashboard/	Android Studio project (Kotlin source, layouts, resources)
simulator/obd_simulator.py	Python OBD data simulator
docs/	Project report, presentation and evaluation form
README.md	Setup and run instructions

11. Conclusion

Smart Car Dashboard demonstrates a functional automotive interaction prototype: a working pipeline from simulated vehicle sensors through MQTT to a live Android display, combined with voice and visual alerts for driver safety. The project addresses the requirements of the Automotive Interaction project by integrating Android development, sensor/actor communication via MQTT, and a clear, glanceable interaction design suitable for use inside a vehicle. Future work includes testing on real vehicle hardware and expanding the evaluation with a larger group of test users.
